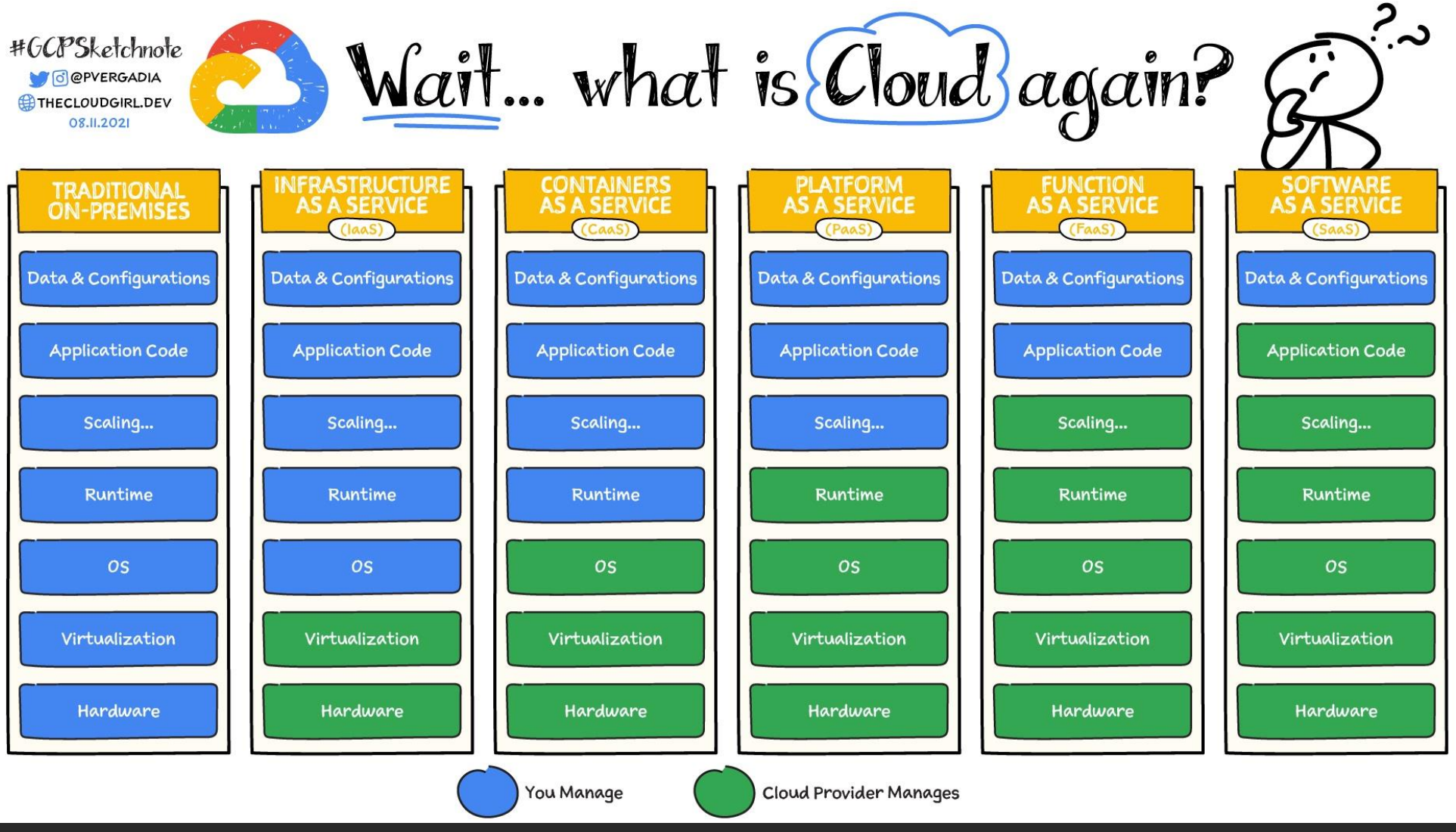
# COMP4442 Service and Cloud Computing
# Tutorial 2

LI Yuhui

Jan 21, 2026

# Quick Review: From Service Model Perspective



# Cloud Computing Frontier

## -- How AI Computing evolves?

# AI Computing: Brief History

- In 2006, CUDA was introduced by NVIDIA.
  - Many cores, suitable for multi-thread computing.
  - GPU is another isolated computer with its own memory, instructions, and ...
- In 2012, Hinton's team utilized NVIDIA GPU to train deep neural network.
  - State-of-the-art performance with non-trivial margin over other models.
  - Milestone for DNN and AI.
- CUDA is designed for graphical rendering, physical system simulation, and scientific computing, not for deep learning itself.

# AI Computing: Brief History

- Deep learning heavily relies on matrix/tensor computation.
- In 2017, NVIDIA proposed Tensor Core
  - Specifically, Matrix-Multiply-Accumulate; Fused Multiply-Add;
    - SIMD; Mixed Precision (INT8, FP16, FP32)
    - Linear Transformation
- In 2017, Transformer; 2018, BERT
- 2019~2022: OpenAI's GPT1, GPT2, GPT3, Scaling Laws, InstructGPT, ChatGPT

# AI Hardware Comparison

- https://docs.cloud.google.com/tpu/docs/system-architecture-tpu-vm?hl=zh-cn

# AI Computing: Number Format

- Beyond Hardware Evolution, Number Format Changed!
  - FP32: 1 sign bit + 8 exponent bits + 23 mantissa bits. It provides a relatively high precision for general - purpose numerical computations in many scientific and engineering applications.
  - FP16: 1 sign bit + 5 exponent bits + 10 mantissa bits. FP16 is widely used in deep - learning accelerators, especially for neural network training and inference on GPUs. It reduces memory footprint and computational requirements at the cost of some precision compared to FP32.

# AI Computing: Number Format

- Beyond Hardware Evolution, Number Format Changed!
  - FP8: 1 sign bit + 4-5 exponent bits + 3-2 mantissa bits (e.g., E4M3/E5M2). It is a family of formats used in modern large-scale AI training and inference to further optimize memory bandwidth and compute efficiency while maintaining accuracy via mixed precision.
  - FP4: 1 sign bit + 2-3 exponent bits + 1-0 mantissa bits (e.g., E2M1). It is an ultra-low-bit format used in cutting-edge hardware for extreme quantization, particularly for high-throughput inference in Large Language Models (LLMs) on edge and data center devices.

# AI Computing: Number Format

- Beyond Hardware Evolution, Number Format Changed!
  - BF16: 1 sign bit + 8 exponent bits + 7 mantissa bits. BF16 was introduced by Intel. It has a similar exponent range to FP32 but a reduced mantissa, which makes it suitable for deep - learning applications where the dynamic range of FP32 is often sufficient, and the reduced precision can speed up operations while still maintaining reasonable accuracy.
  - TF32 (19 bit valid): 1 sign bit + 8 exponent bits + 10 mantissa bits. TF32 is a format used by NVIDIA's Tensor Cores. It offers a balance between the precision of FP32 and the efficiency of lower - precision formats like FP16. It can significantly accelerate matrix multiplication operations in deep - learning frameworks.

# AI Computing: Number Format

- What if we don't use low precision floating points?

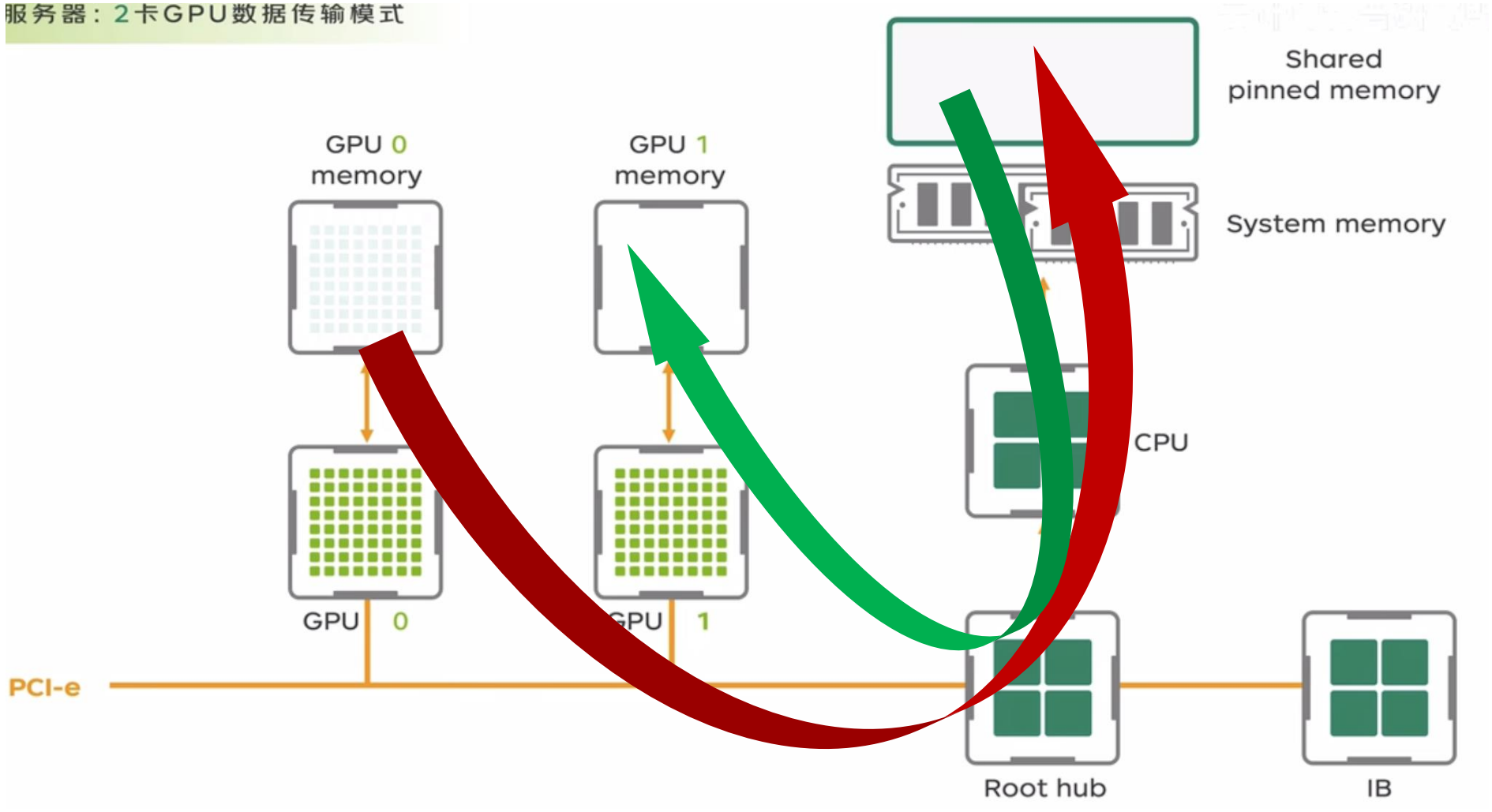
| 型号 | 架构 | 制程 | 接口类型 | 显存(GB) | 带宽(TB/s) | FP32算力(TFLOPS) | FP16算力(TFLOPS) | 典型功耗(W) | 定位 | 价格(万元) |
|---|---|---|---|---|---|---|---|---|---|---|
| **H100 SXM5** | Hopper | 4nm | SXM5 | 80(HBM3) | 3.35 | 60 | 1,344 | 700 | 旗舰级训练 | 28-35 |
| **H100 PCIe** | Hopper | 4nm | PCIe 5.0 | 80(HBM3) | 2.0 | 51.2 | 989 | 350 | 高端训练/推理 | 18-22 |
| **H200** | Hopper+ | 4NP | SXM5 | 141(HBM3e) | 4.8 | 72 | 2,000+ | 700+ | 超大模型训练 | 40-50 |
| **RTX 5090** | Blackwell | 4nm | PCIe 5.0 | 32(GDDR7) | 1.5 | 104.8 | 419 | 450 | 高端消费级 | 1.8-2.2 |
| **RTX 4090** | Ada | 4nm | PCIe 5.0 | 24(GDDR6X) | 1.0 | 82.6 | 330 | 450 | 主流消费级 | 1.2-1.5 |

# AI Computing: Scaling Up!!!

- GPU is installed on PCIe slot.
  - Most of the devices are installed on PCIe: NIC, SSD, Wi-Fi, Audio Card.
- So far, we only focus on single-node GPU evolution.
- Now, look at the scale of LLM– we must connect GPUs to scale up!
- Ways of Scaling:
  - Single Node, Multiple GPUs.

# AI Computing: Scale Up on a Single Node

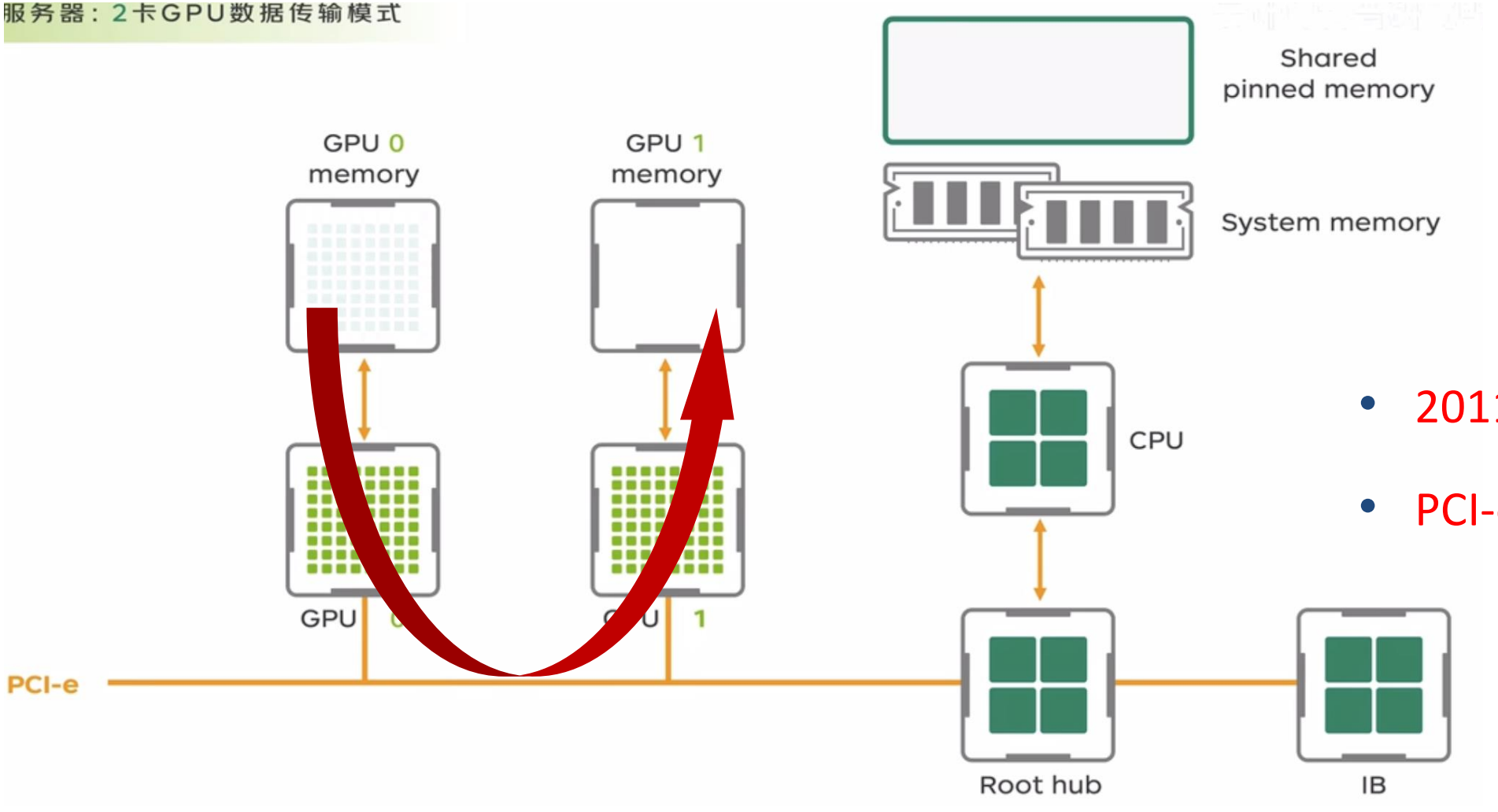
- Single Node, 2 GPU




- CPU Involved!
- Data Copy!

# AI Computing: Scale Up on a Single Node

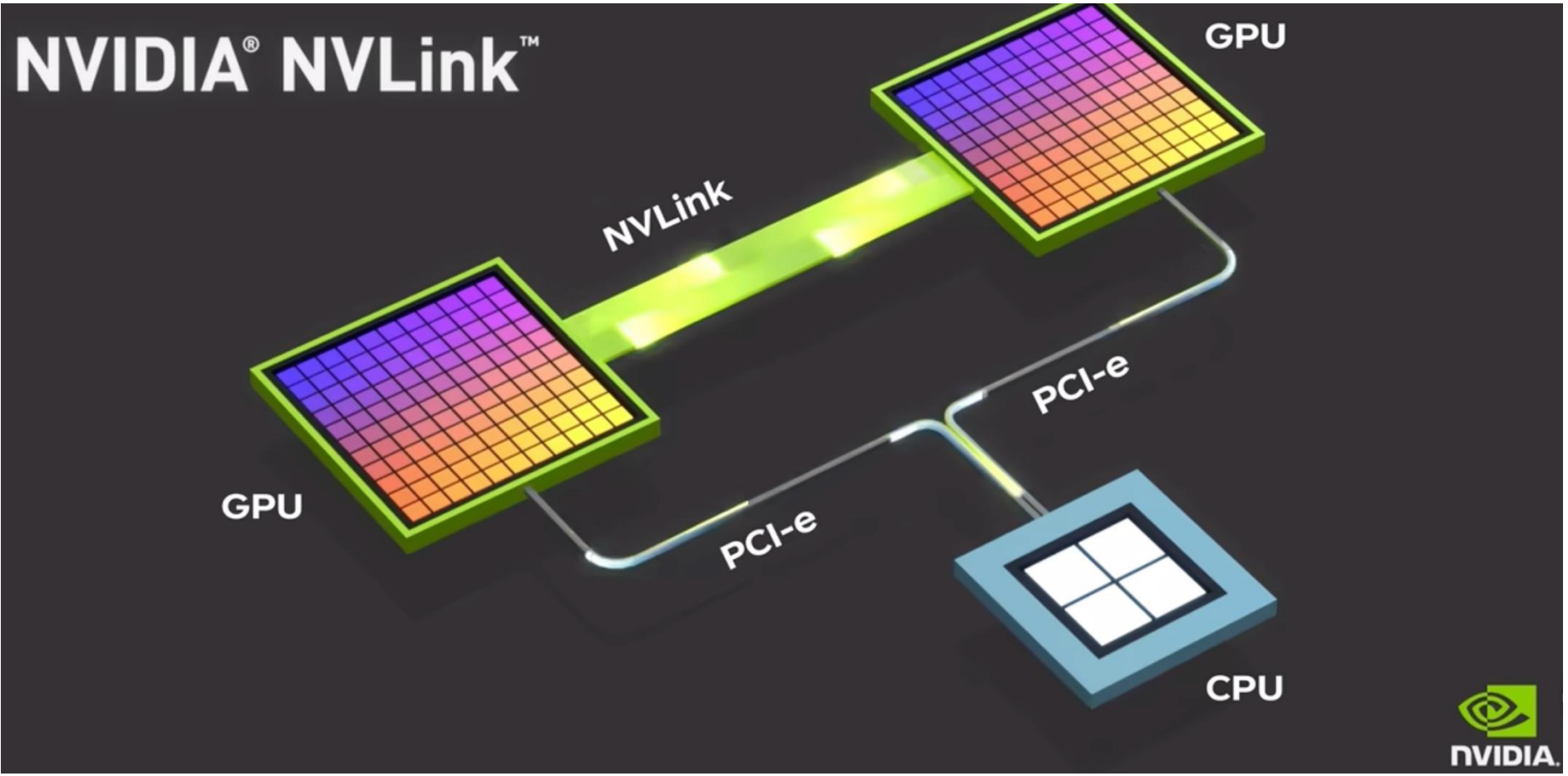
- Single Node, 2 GPU




- 2011: NVIDIA GPU Direct
- PCI-e bottleneck!

# AI Computing: Scale Up on a Single Node

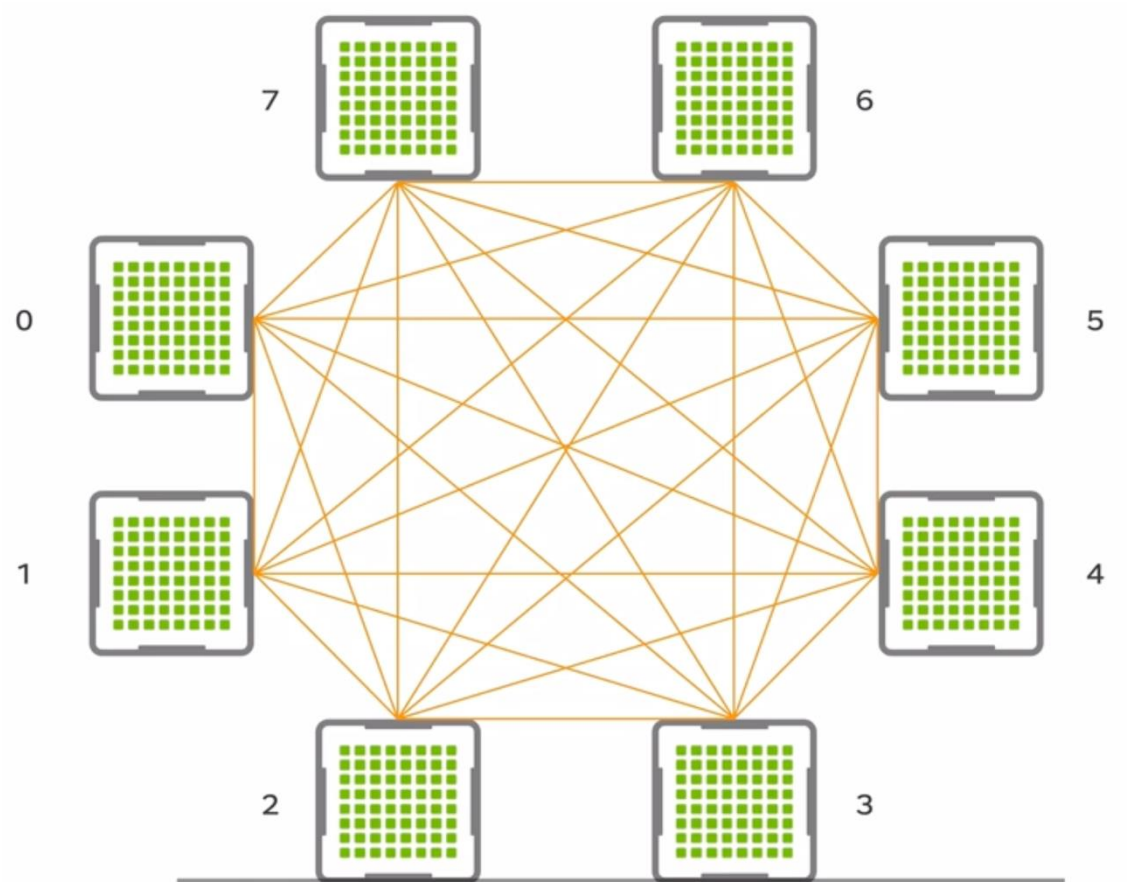
- Single Node, 2 GPU




- 2014: NVIDIA NVLink
- Bypass PCIe!
- #PCIe slots bottleneck!

# AI Computing: Scale Up on a Single Node

- Single Node, 8 GPU?




- Full-Mesh GPU NVLink
- 8 GPUs: 28 #NVLinks
- 16 GPUs: 120 #NVLinks

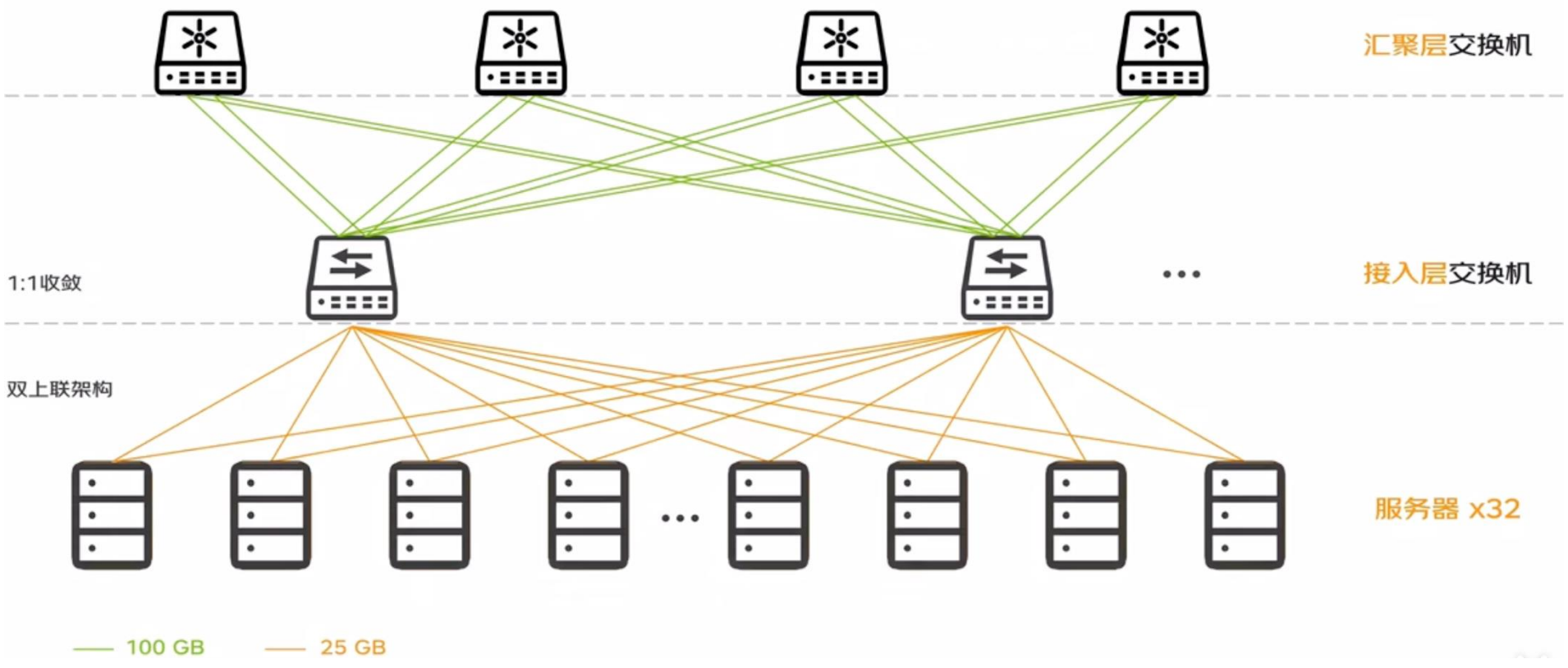
- Scaling needs to be extended across nodes.

# AI Computing: Scale Up on on a Single Node

- Remember! Computer network is always a good way to scale up!
  - Much slower than hardware-level scale up.
  - Rely on network protocol and OS-kernel packet processing.
  - Still, it is the most common way to scale up your computing resources to any scale.
- NVIDIA also follows the same principle. NVLink ---> NVSwitch.

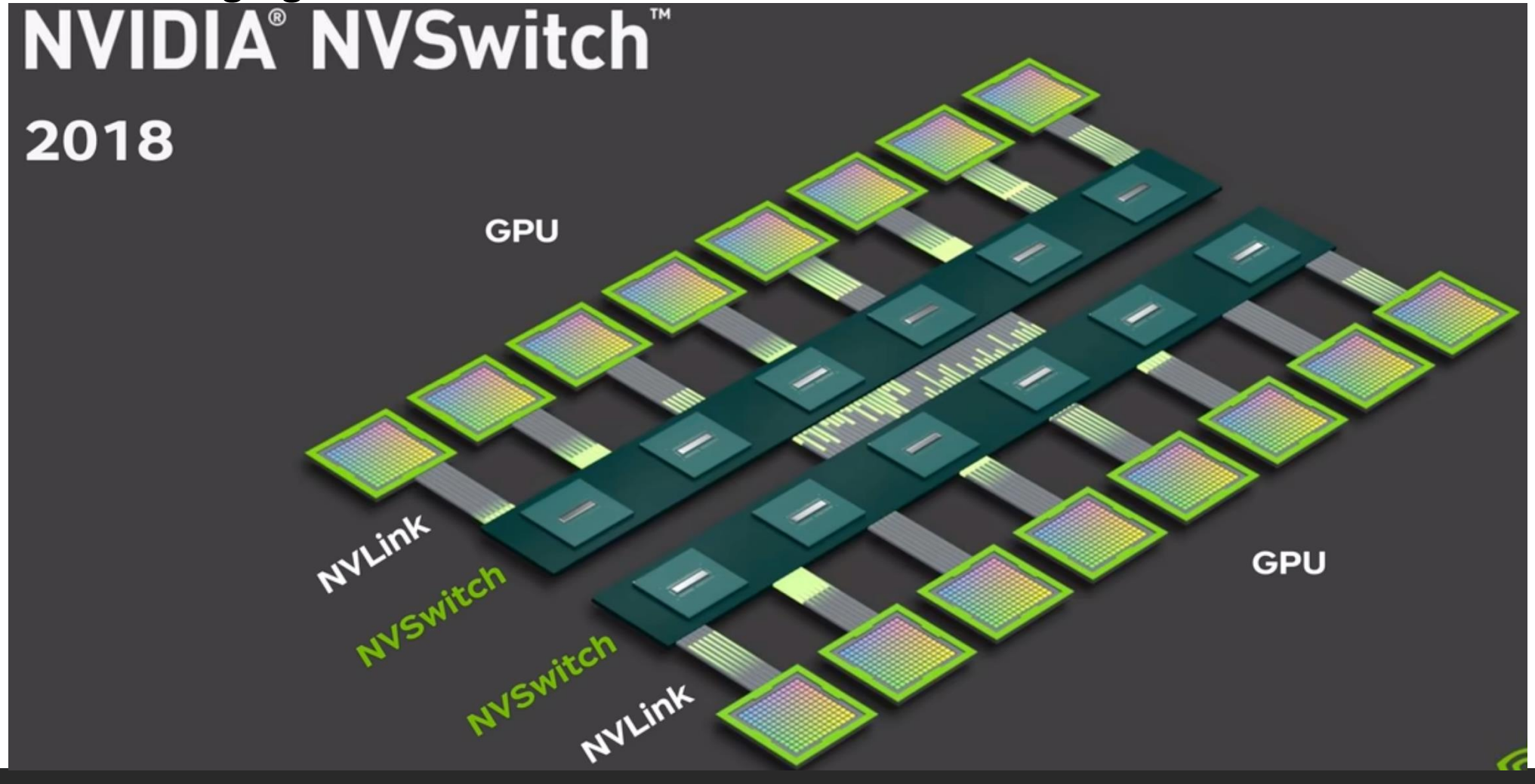
# AI Computing: Scale Up on a Single Node

- In fact, in data center, high throughput and low latency networking technology is matured.
- Multi-level Switch.



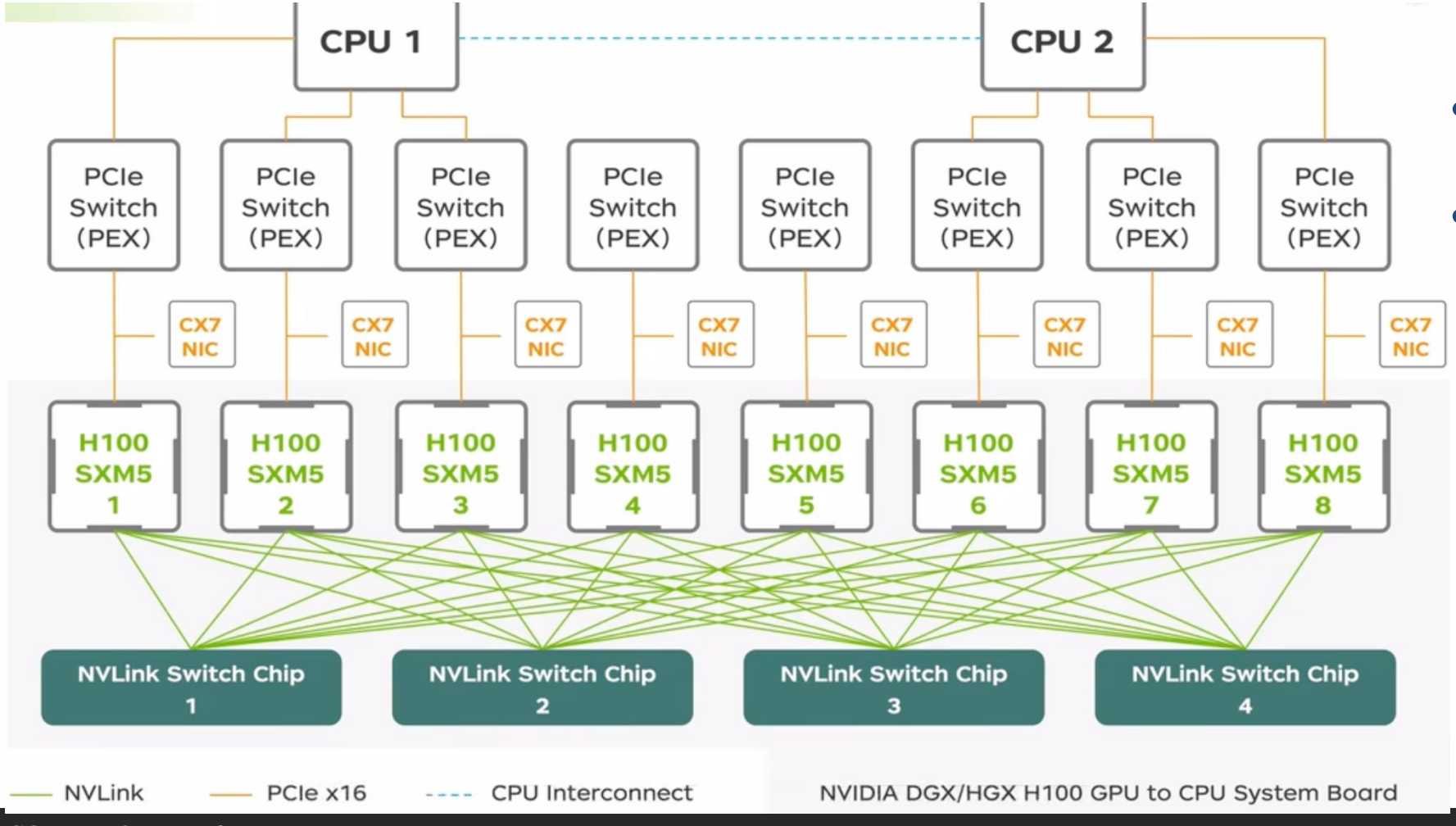
# AI Computing: Scale Up on a Single Node

- NV Switch: Bridging NVLink-enabled multi-GPUs node.



# AI Computing: Scale Up on a Single Node

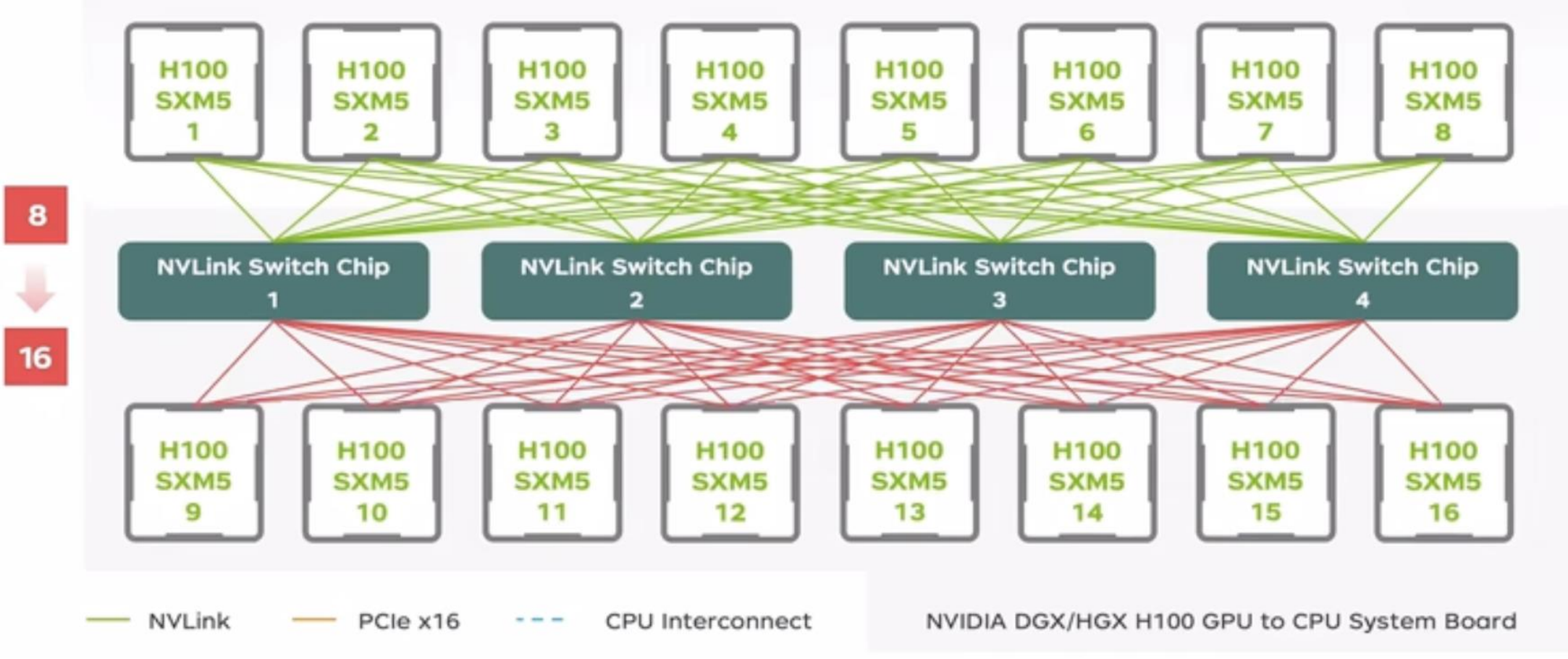
- NV Switch: Bridging NVLink-enabled multi-GPUs node.




- More NVLink??????
- What are the advantages?

# AI Computing: Scale Up on a Single Node

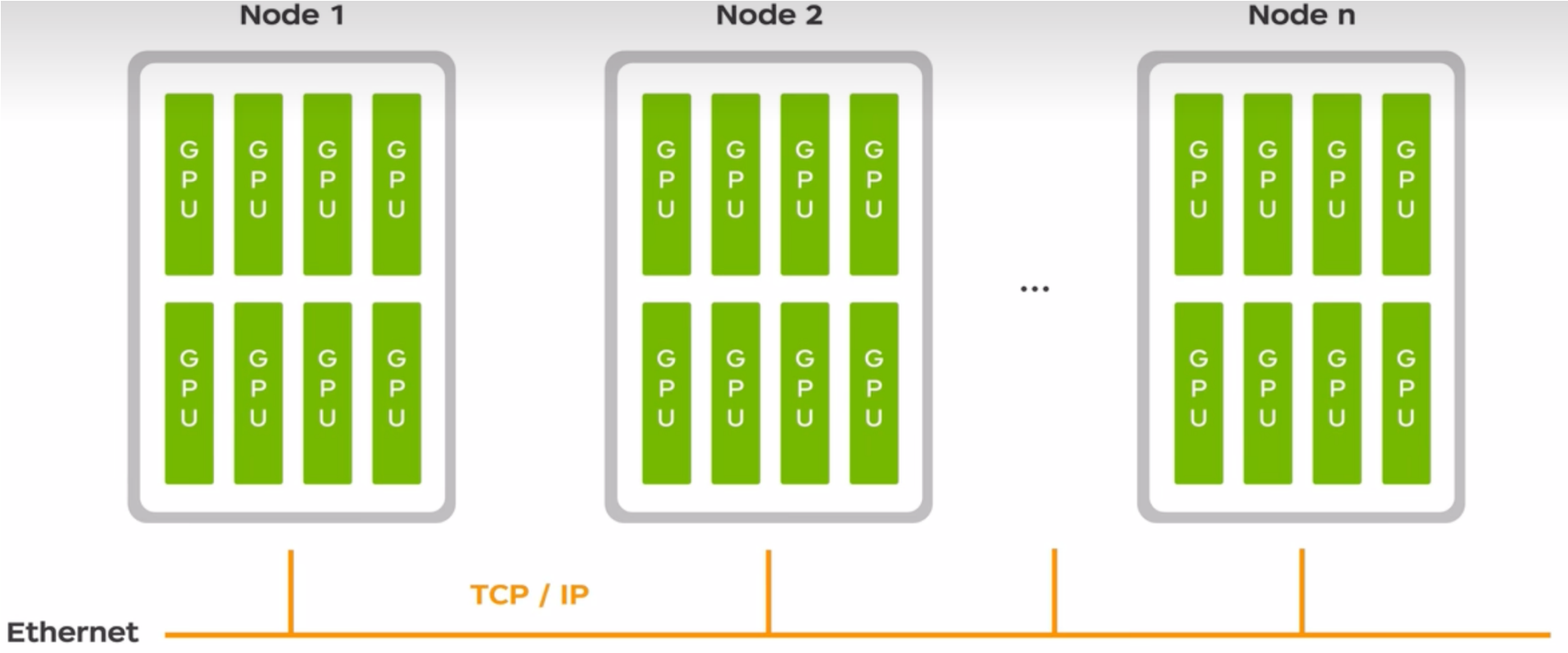
- NV Switch: Bridging NVLink-enabled multi-GPUs node.




- Bottleneck: From GPU's #NVLink ===> Switch Chip!

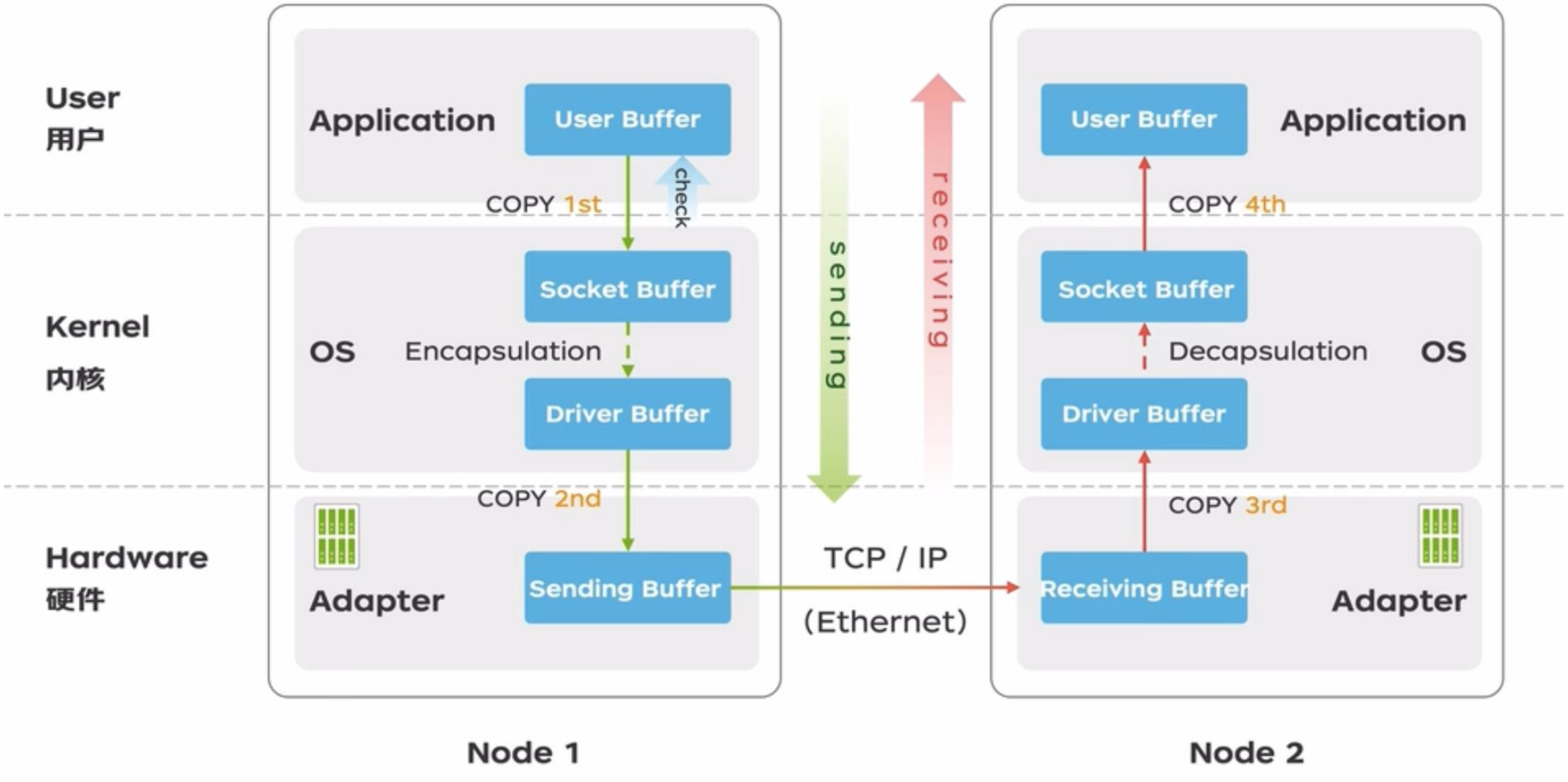
# AI Computing: Scale Up to Multi Nodes

- Inter-connection of multiple nodes: Ethernet



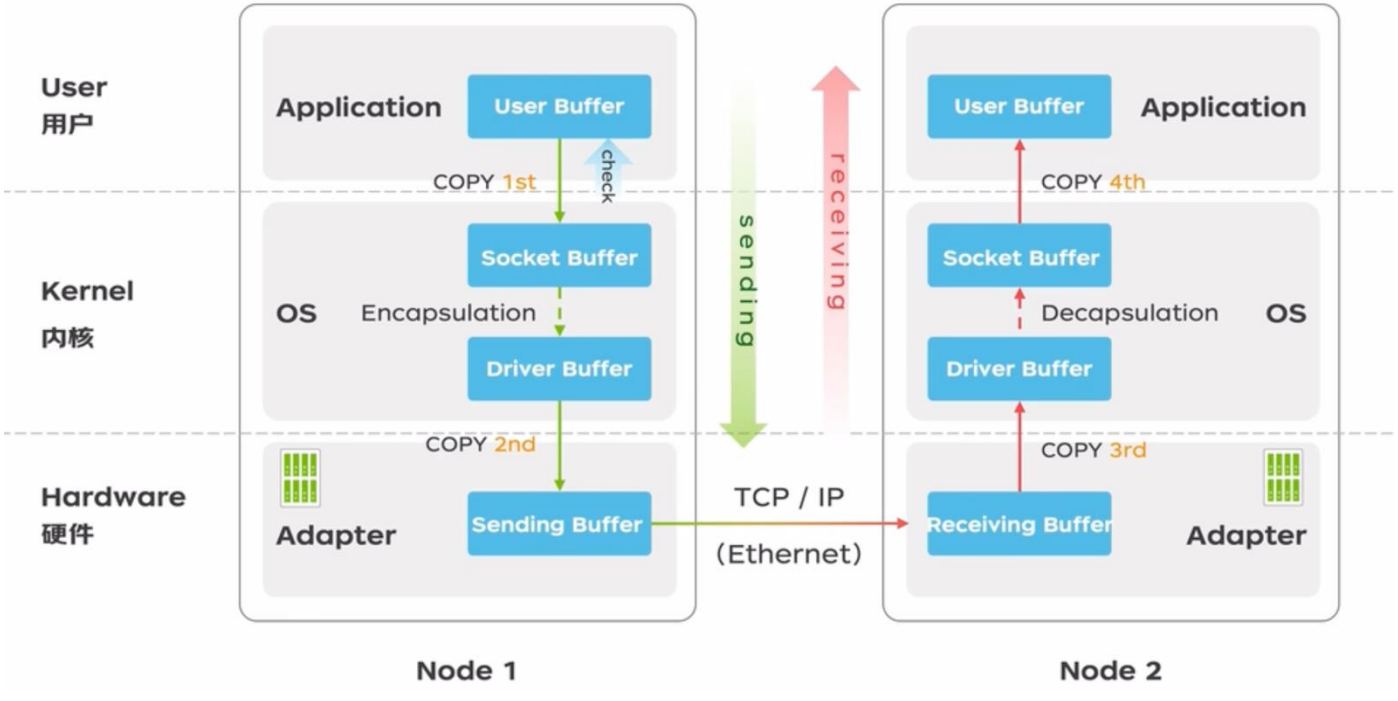
# AI Computing: Scale Up to Multi Nodes

- Inter-connection of multiple nodes: Ethernet + TCP/IP



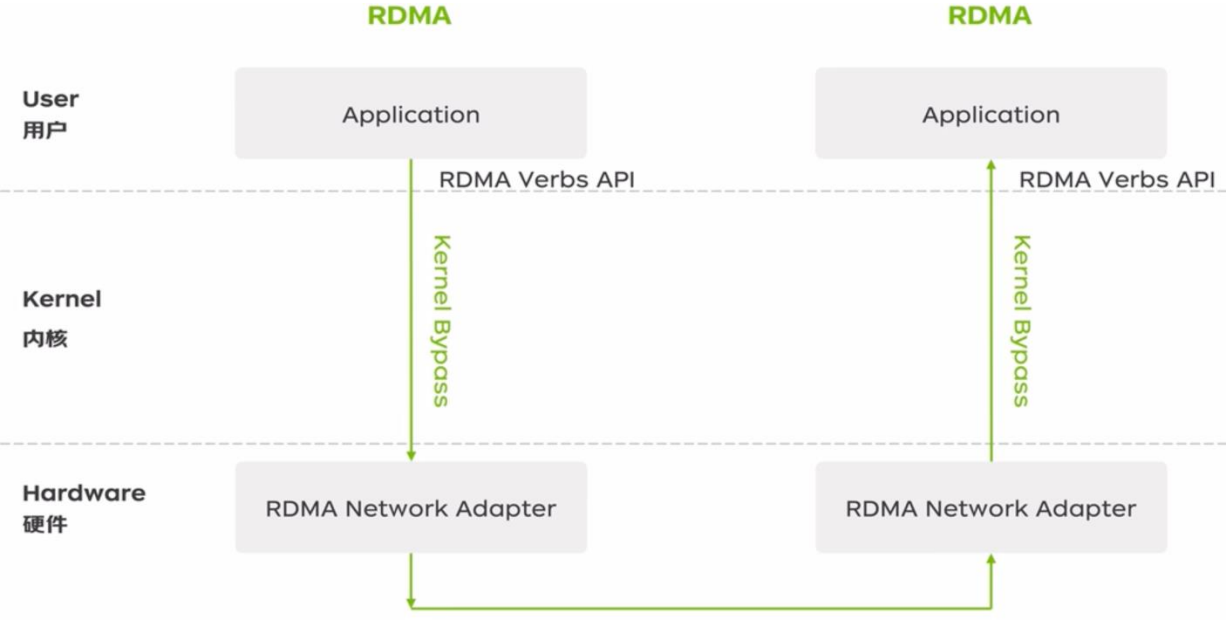
# AI Computing: Scale Up to Multi Nodes

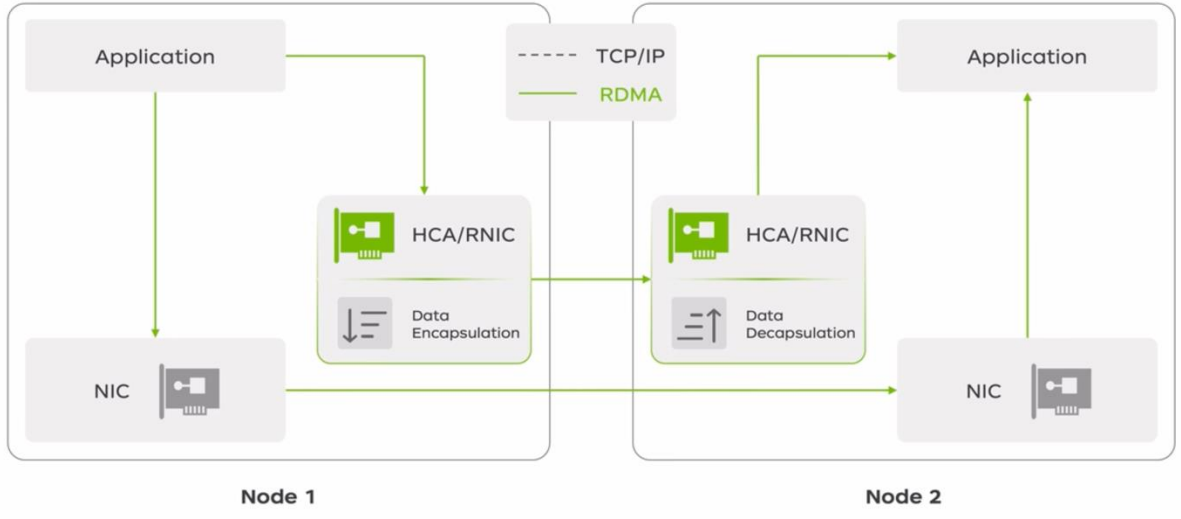
- Solution?
- Recall: How to deal with low speed external devices such as HDD, printer?
  - We use Direct Memory Access (DMA) to bypass CPU!



# AI Computing: Scale Up to Multi Nodes

- Remote Direct Memory Access (RDMA):

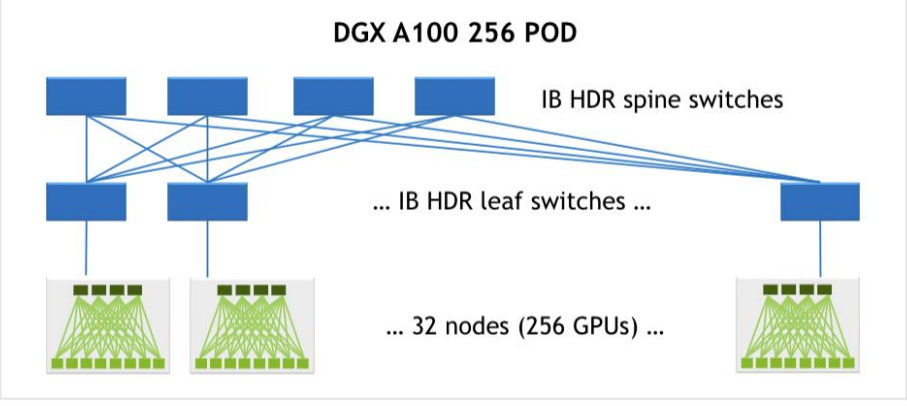


# AI Computing: Scale Up to Multi Nodes

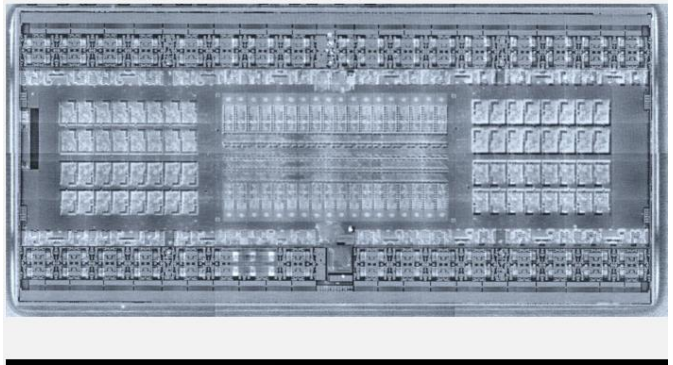
- SuperNode!

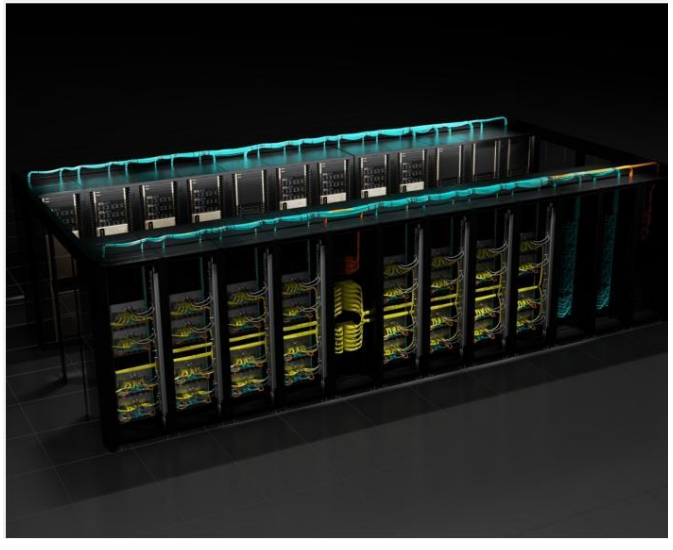
## SCALE-UP WITH NVLINK NETWORK



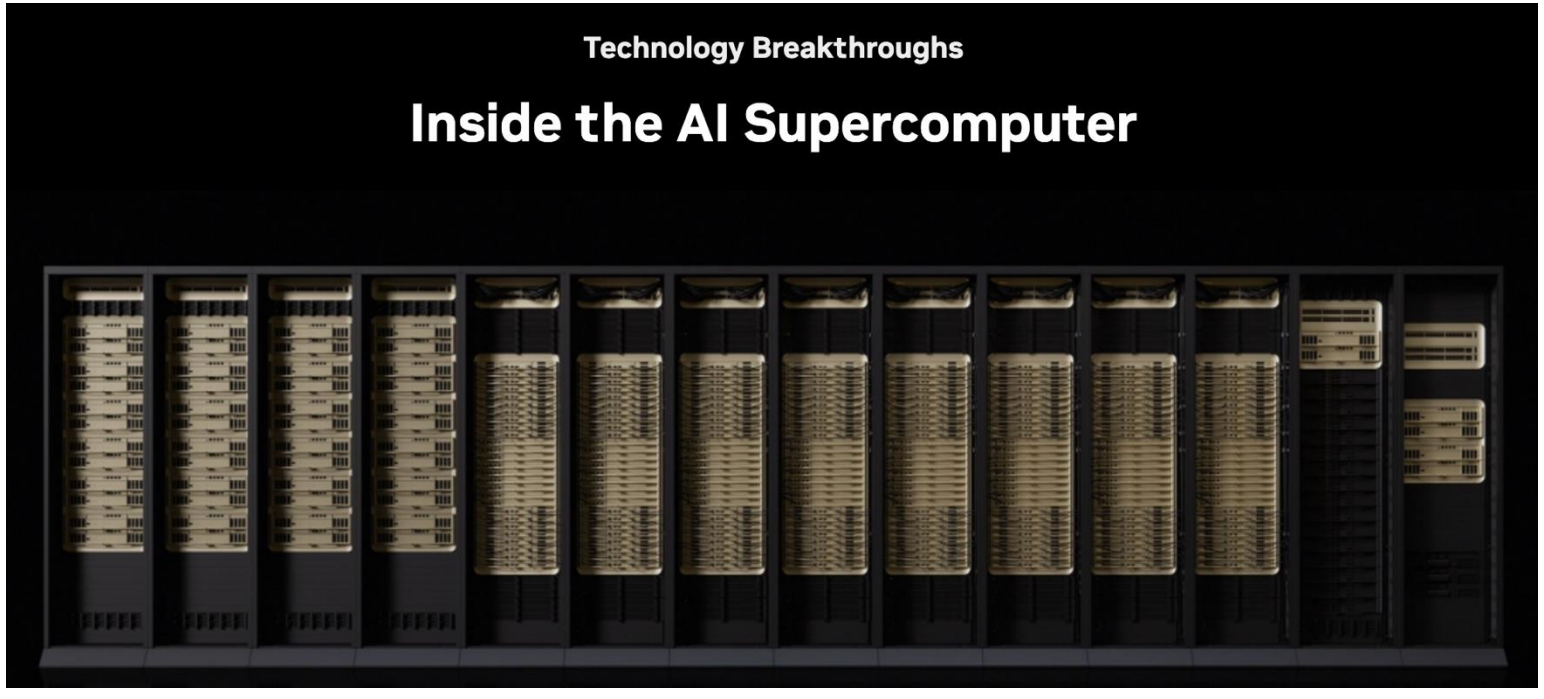
| | A100 SuperPOD | | | H100 SuperPOD | | | Speedup | |
|---|---|---|---|---|---|---|---|---|
| | Dense PFLOP/s | Bisection [GB/s] | Reduce [GB/s] | Dense PFLOP/s | Bisection [GB/s] | Reduce [GB/s] | Bisection | Reduce |
| 1 DGX / 8 GPUs | 2.5 | 2,400 | 150 | 16 | 3,600 | 450 | 1.5x | 3x |
| 32 DGXs / 256 GPUs | 80 | 6,400 | 100 | 512 | 57,600 | 450 | 9x | 4.5x |

# How it looks like?





14 x 72 = 1008




During the pre-training stage, training DeepSeek-V3 on each trillion tokens requires only 180K H800 GPU hours, i.e., 3.7 days on our cluster with 2048 H800 GPUs. Consequently, our pre-

# AI Computing: Scale Up to Multi Nodes

- Large Scale AI Infrastructure are very rare.
  - 10, 000 + NVIDIA H100 80GB GPU
  - Only the leading cloud computing service providers have.
  - Scaling the AI computing power is also a mirror of scaling general purpose computing power.
    - Super computer.
- With computing infrastructure's scale expansion, we definitely need better management!

# Cloud Computing Basic

## -- How to manage large scale infrastructure?

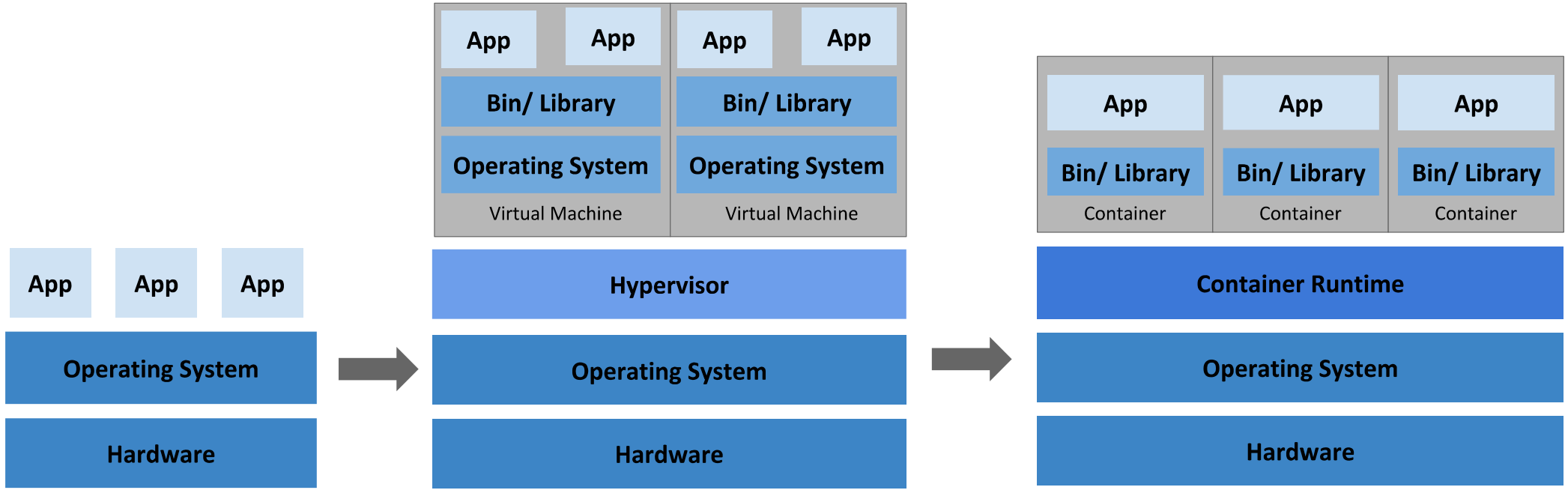
# Cloud Computing: Brief History

- 2004~2007: Google developed cgroups technique.
  - Short for Control Groups.
  - Grouping resources and control visible resources for running processes.
- 2008: cgroups merged into Linux kernel.
- 2013: Docker
- 2014: Kubernetes
- 2015: Cloud Native Computing Foundation

# Cloud Computing: Brief History

- 2015-2016: Competing for Container Orchestration Standard
  - Docker Swarm; Apache Mesos; Kubernetes
- 2017: Kubernetes won the competition as docker swarm gradually stopped its maintenance.
- 2018: Cloud Native

# Standalone, VM, and Container




**Traditional Deployment** **Virtualized Deployment** **Container Deployment**

- Software Lifecycle: Develop, Maintenance, Deployment...
- Hardwares = Packs, Containers = Items, Resource Allocation ---> Knapsack Problem
- How to update your software version without service downtime?
- How to scale up your service throughput when needed?

# Cloud Computing: Kubernetes

- From infrastructure providers perspectives:
  - We need an automatic container assignment program.
- For users' perspectives:
  - We don't care about how you deploy applications, but we care about QoS.
- In essence, Kubernetes is a container orchestration platform.
- Also known as k8s. (K ubernete [eight characters] s)

# Cloud Computing: Landscape

https://landscape.cncf.io/

# Resource Pooling & Reconfiguration

- Despite the paradigm changes, the essence of computing remains stable.
- Previously, we need a physical machine (bare metal server).
  - Resources is pre-configured.
- Now, we need to construct a “computer” from many distributed servers.
  - With Kubernetes as a “Operating System”.
  - Upload your programs and resource claims.
  - Kubernetes allocate resources and run you program!

# Another Angle: Elements of Computing

- What key components do a computer have?
  - Computing Resources
    - CPU, GPU, FPGA, ASIC
    - Memory (e.g., HBM, NUMA, UMA)
  - Networking Resources
    - Networking Devices, IP Address, Protocol
    - Bandwidth, Latency (QoS)
  - Storage Resources
    - Host File System, Layered File System, Network File System (Hadoop)
  - I/O Resources

# Virtualize Everything

- Computing Virtualization
  - cgroups, container runtime, docker, VM
- Network Virtualization
  - NFV, SDN
- Storage Virtualization
  - NFS